Más allá del Prompt Injection

LLM08 contra Sistemas de Embeddings

Carles Cano & Pol Centelles

HACKBCN
2026

Índice

01 Introducción

02 Fundamentos Técnicos

03 LLM08: Vector and Embedding

04 PoCs – Demostraciones Prácticas

05 Conclusiones

01

Introducción

Contexto, objetivos y OWASP LLM Top 10

Whoami

Pol Centelles

Cybersecuirty Analyst

Experiencia

- 1 año en seguridad de redes
- 4 años como Cybersecurity Analyst
- OSCP / CARTP / CRTE

Especialización

- Seguridad web y Red Team

@PolEspurnes

Carles Cano

Cybersecurity Delivery Consultant

Experiencia

- 4 años en desarrollo
- 5 años como professional de la ciberseguridad
- 1 año como Cybersecurity Delivery Consultant

Especialización

- Seguridad en dispositivos móviles iOS y Android.

@K4ine

Wise Security (Var Group)

Barcelona, España

Objetivos de la charla

01 Funcionamiento de los LLMs y RAGs

02 Vulnerabilidad LLM08

03 Demostraciones Prácticas

04 Mitigaciones y Recomendaciones

Meta principal: Explicar la vulnerabilidad LLM08 y sus implicaciones de seguridad en sistemas basados en embeddings.

OWASP LLM Top 10 2025

01 Prompt Injection

02 Sensitive Info Disclosure

03 Supply Chain

04 Data and Model Poisoning

05 Improper Output Handling

06 Excessive Agency

07 System Prompt Leakage

08 Vector and Embedding Weaknesses ⭐
Flaws en implementaciones RAG: embeddings, vector DBs, retrieval

09 Misinformation

10 Unbounded Consumption

02

Fundamentos Técnicos de LLMs

Tokens, embeddings, probabilidad y arquitecturas RAG

¿Qué son los LLMs?

*LLMs (Large Language Models) son modelos de inteligencia artificial **entrenados** con grandes cantidades de texto para comprender y generar **lenguaje natural** de manera coherente y contextual.*

Ejemplos de Integraciones

- Asistentes virtuales y chatbots
- Generación automática de contenido
- Búsqueda inteligente y sistemas de recomendación
- Automatización de tareas administrativas

¿Cómo funcionan?

01 Tokenización

Los LLMs NO leen palabras, leen tokens

```
En el contexto de los LLMs, un token es la unidad mínima de texto que el modelo
 utiliza para procesar y generar lenguaje.
```

121 chars → 31 tokens

¿Cómo funcionan?

01 Tokenización

*Cada **token** tiene asignado un **tokenID**, que el modelo asocia a un **embedding**.*

Token String	Token ID	Embedded Vector
"The"	101	[0.5, 0.8, ..., 0.65]
"cat"	2598	[0.1, 0.3, ...-0.82]
"sat"	4450	[0.41, -0.53, ..., 0.8]
."	119	[0.17, 0.75, ..., 0.2]

¿Cómo funcionan?

02 Embeddings Vectoriales

Conjunto de números que representan el significado semántico de un token.

	King	Queen	Woman	Princess
Royalty	0.99	0.99	0.02	0.89
Masculinity	0.99	0.05	0.01	0.02
Femininity	0.05	0.93	0.99	0.94
Age	0.7	0.6	0.5	0.1
...	⋮	⋮	⋮	⋮

No son números aleatorios, sino

coordenadas en un espacio de representaciones semánticas.

Retrieval-Augmented Generation (RAG)

*Es una técnica que combina modelos de lenguaje (LLMs) con sistemas de búsqueda o **recuperación de información** para producir **respuestas más precisas** y contextualizadas.*

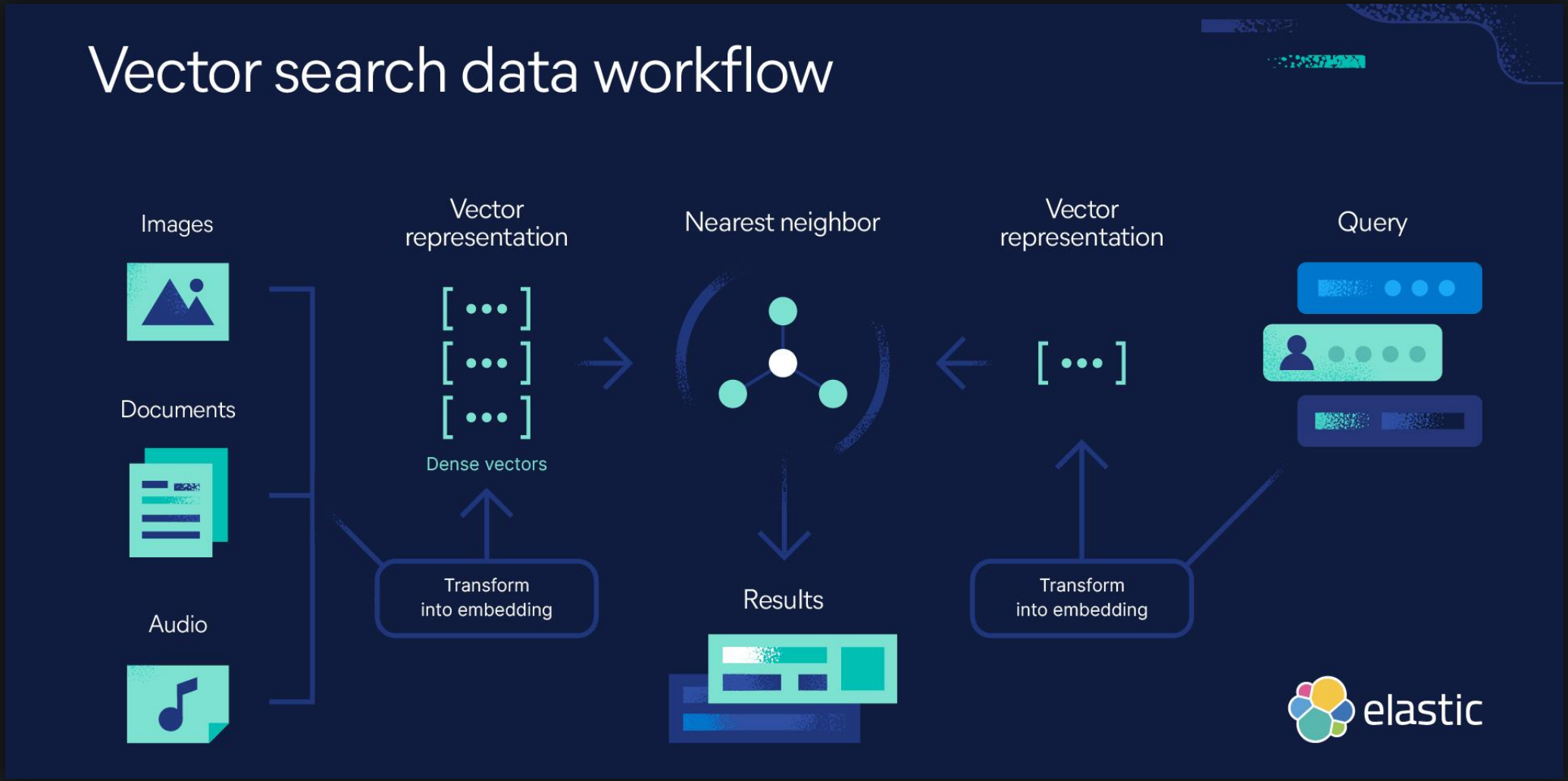
Beneficios

- ✓ Conocimiento actualizado
- ✓ Reducción de alucinaciones
- ✓ Datos privados

Componentes Clave

- Embedding Model
- Vector DB
- Retriever
- LLM

Retrieval-Augmented Generation (RAG)



What are vector embeddings?
elastic.co

¿Cómo funcionan?

Similitud entre Vectores

Fórmula de similitud:

$$similarity = \cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$$

Si similarity > threshold → BLOCK

Mensaje del usuario	Similitud	¿Bloqueado?
Cómo hacer algo ilegal	0.95	✅
Cómo cometer un acto ilegal	0.90	✅
Cómo hacer cosas que no están permitidas	0.75	⚠️
Historia de alguien que aprende algo ilegal	0.55	❌

03

LLM08: Vector and Embedding Awareness

Análisis profundo de la vulnerabilidad

LLM08: Definición según OWASP

08

Vector and Embedding Weaknesses

OWASP LLM Top 10 - 2025

"Vulnerabilidades derivadas de las deficiencias en cómo los vectores y embeddings son generados, almacenados o recuperados, dando lugar a que se pueda **inyectar contenido dañino**, **manipular salidas del modelo** o **acceder a información sensible**."

Contextos de aparición

- Sistemas RAG
- Memoria de LLMs
- Búsquedas semánticas
- Sistemas de recomendación

OWASP - LLM08:2025
genai.owasp.org

¿Cómo aparecen estas vulnerabilidades?

Falta de Controles de Acceso
Impacto: Data leaks, fallos de autorización

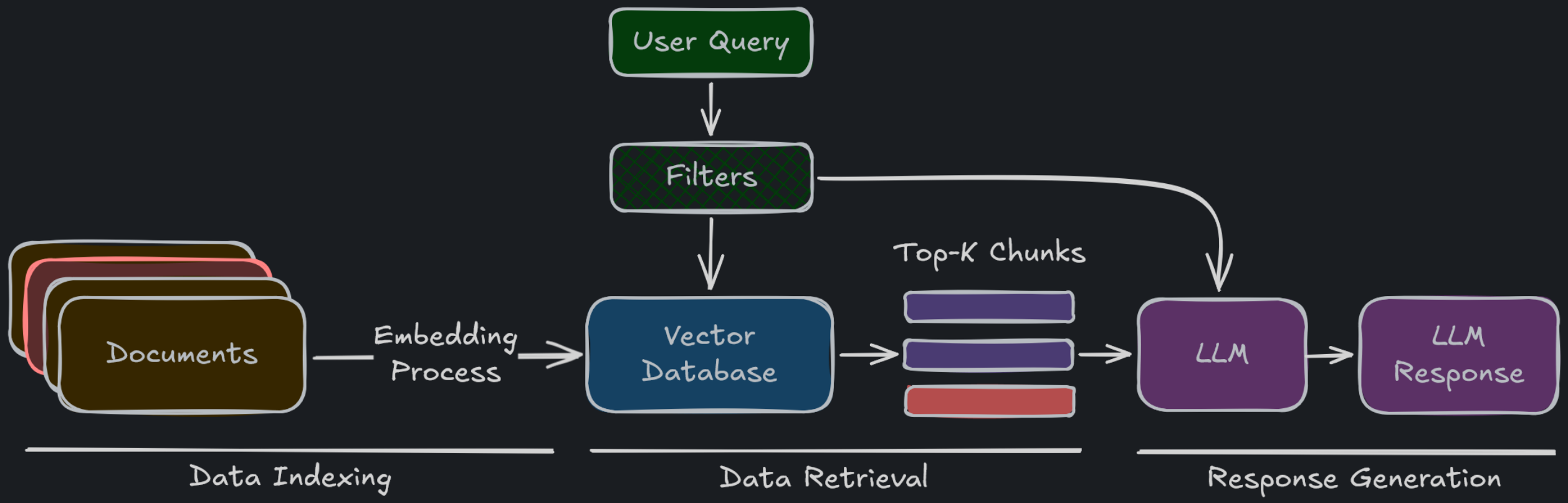
Separación Inadecuada de Tenants
Impacto: Violaciones de privacidad, daño reputacional

Validación Insuficiente de Fuentes
Impacto: RAG poisoning, behavior alteration

Asunción de Seguridad por Diseño
Impacto: Embedding Inversion, data leaks

Problema clave: Se están protegiendo los **prompts**, pero ignorando los **vectores**.

Superfície de ataque: RAG Pipeline



04

PoCs - Demostraciones Prácticas

Tres ataques demostrados con código y análisis técnico

Introducción a las PoCs

Objetivo de las PoCs

Demostrar que los ataques a embeddings no son teóricos con ejemplos de vectores de ataque **realistas**.

Estructura de cada PoC

- Escenario planteado
- Flujo de ataque
- Vulnerabilidad
- Mitigaciones

Repositorio

GitHub - PolEspurnes/PoCs_LLM08_HackBCN: PoCs creadas para la presentación de LLM08 en HackBCN 2026
GitHub

PoCs_LLM08_HackBCN

Introducción a las PoCs

3 Demostraciones

01 RAG Poisoning

02 Evasión de Controles Semánticos

03 Fallos de Autorización en RAGs

Embedding Model

sentence-transformers/all-MiniLM-L6-v2
Modelo estándar en RAG systems. 384-dimensional embeddings.

Vector Database

FAISS
Facebook AI Similarity Search. Librería eficiente para búsqueda vectorial.

Python Flask App

Flask
Framework simple para aplicaciones web y REST

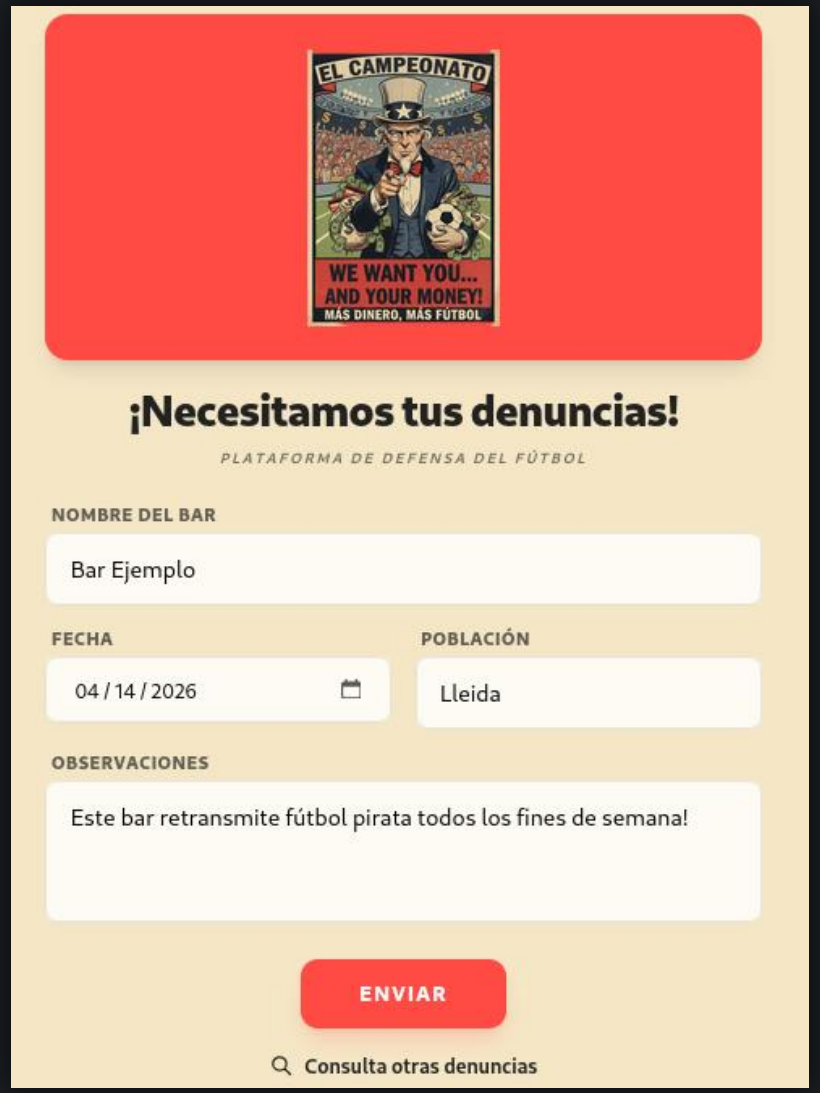
LLM

OpenRouter
API unificada para múltiples LLMs.

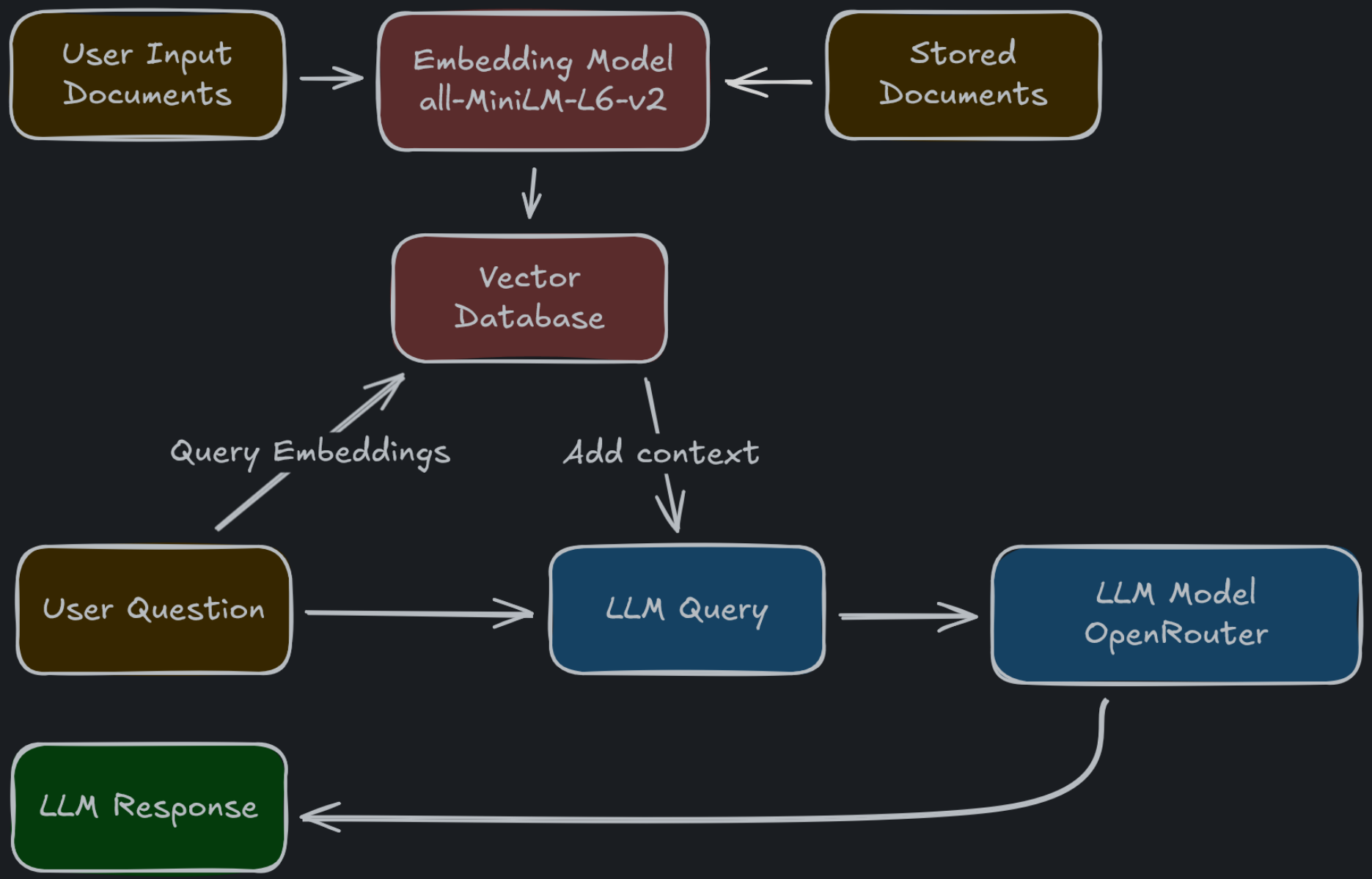
Introducción a las PoCs

```
[PoCs_LLM08_HackBCN]$ python3 main.py
 * Serving Flask app 'main'
 * Running on http://127.0.0.1:5000
```

PoC #1: RAG Poisoning



PoC #1: Escenario



PoC #1: Fujo de ataque

Flujo del Ataque

STEP 01

1. Documento Malicioso

Atacante envía un documento con instrucción oculta.

↓

STEP 02

2. Ingesta & Embedding

El embedding model (all-MiniLM-L6-v2) convierte todo el documento a vectores.

↓

STEP 03

3. Vector Database

Los embeddings envenenados se almacenan junto a los legítimos en FAISS. Indistinguibles de vectores normales.

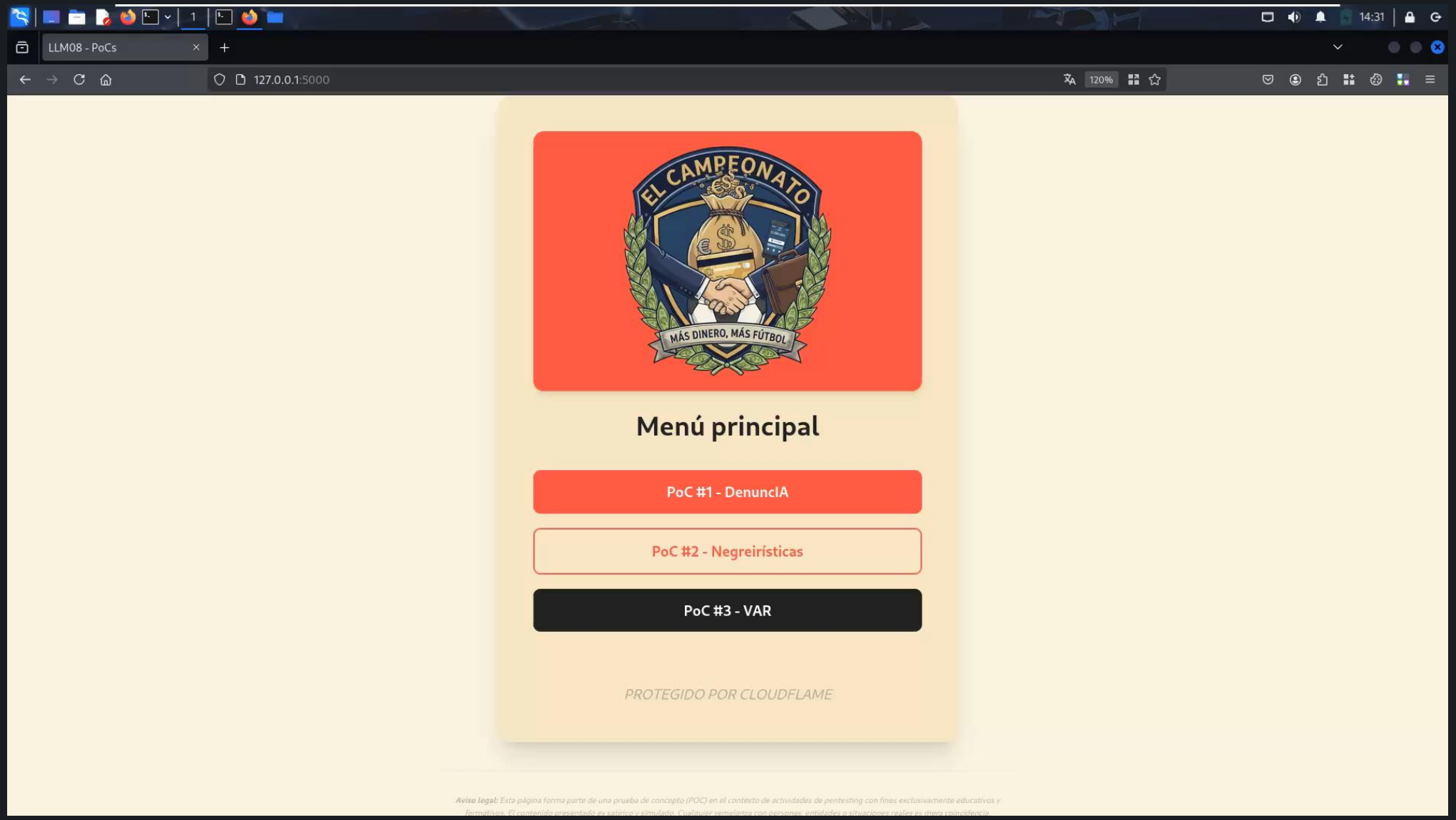
↓

STEP 04

4. Retrieval & Injection

El documento envenenado es recuperado por similitud semántica. Payload se inyecta en el contexto y se envía al LLM.

PoC #1: Ataque


LLM08 - PoCs
127.0.0.1:5000
EL CAMPEONATO
MÁS DINERO, MÁS FÚTBOL
Menú principal
PoC #1 - DenuncIA
PoC #2 - Negreirísticas
PoC #3 - VAR
PROTEGIDO POR CLOUDFLAME

PoC #1: ¿Dónde está la vulnerabilidad?

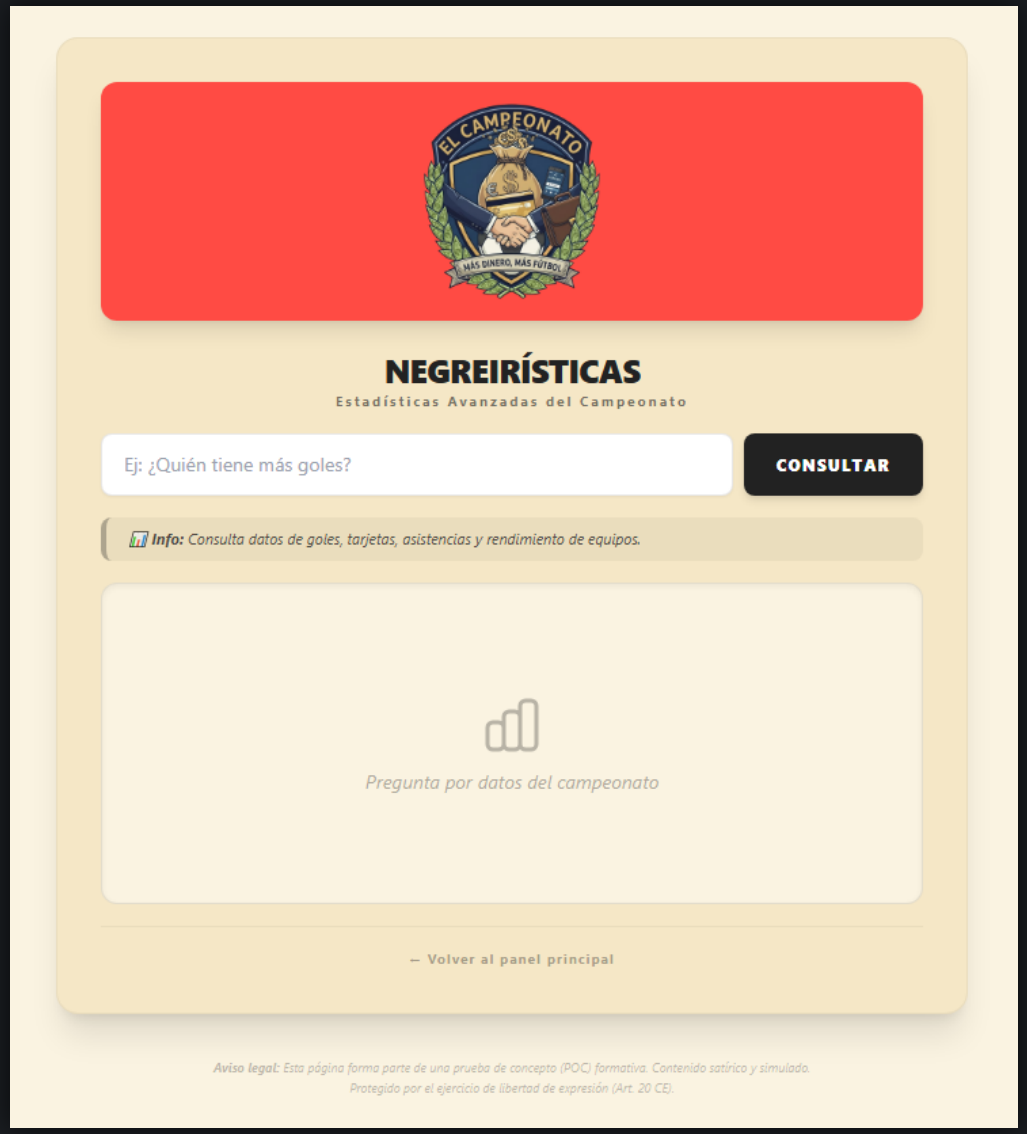
El modelo de embeddings no distingue entre contenido legítimo e instrucciones maliciosas.
Ambos se convierten en vectores.

1. Preservación en RAG
2. Confianza en Contexto
3. Ausencia de Sanitización

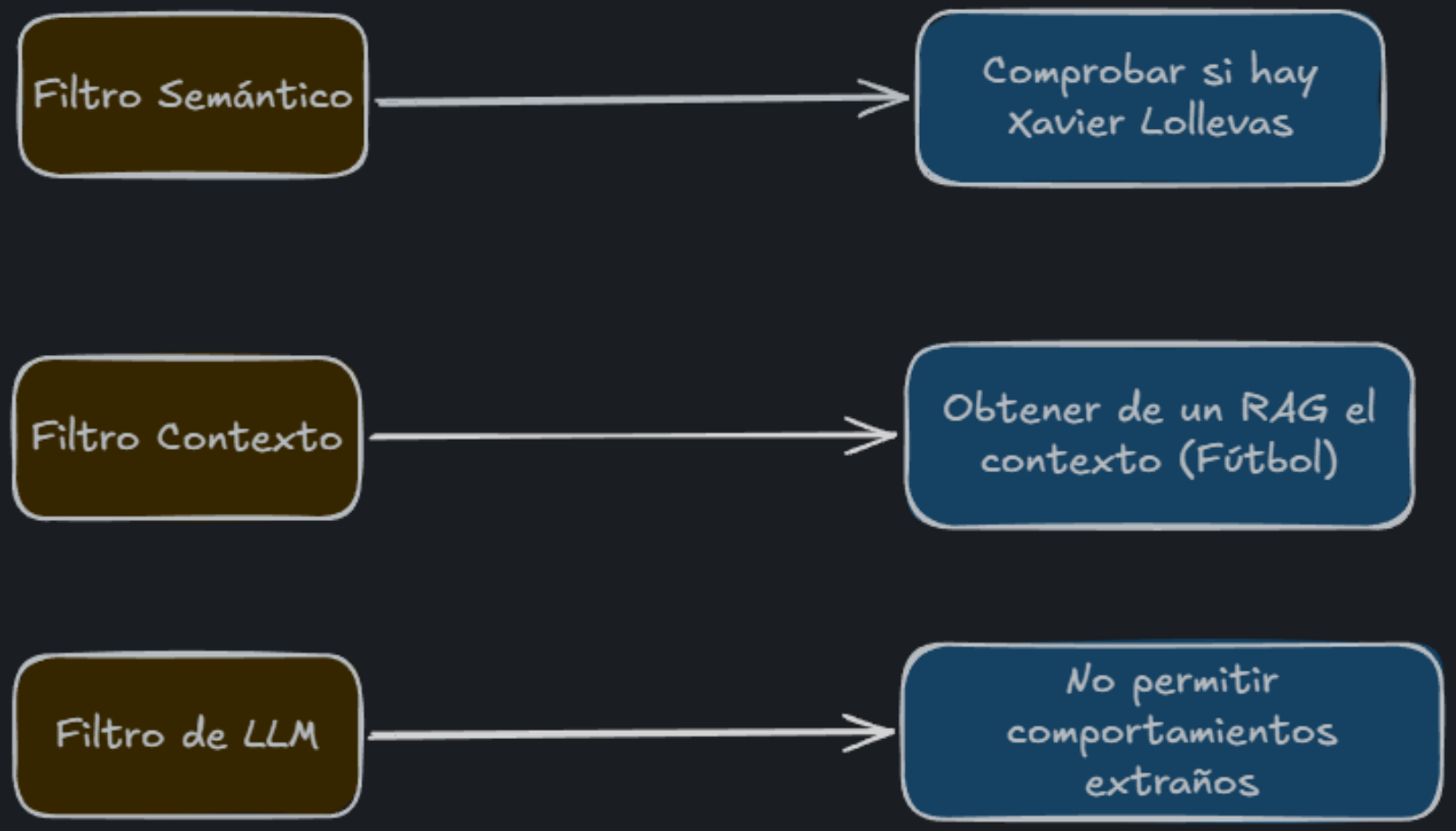
PoC #1: Mitigaciones

- Validación Pre-Ingesta
- Sandboxing de contexto
- Control de confianza
- Monitorización y Análisis

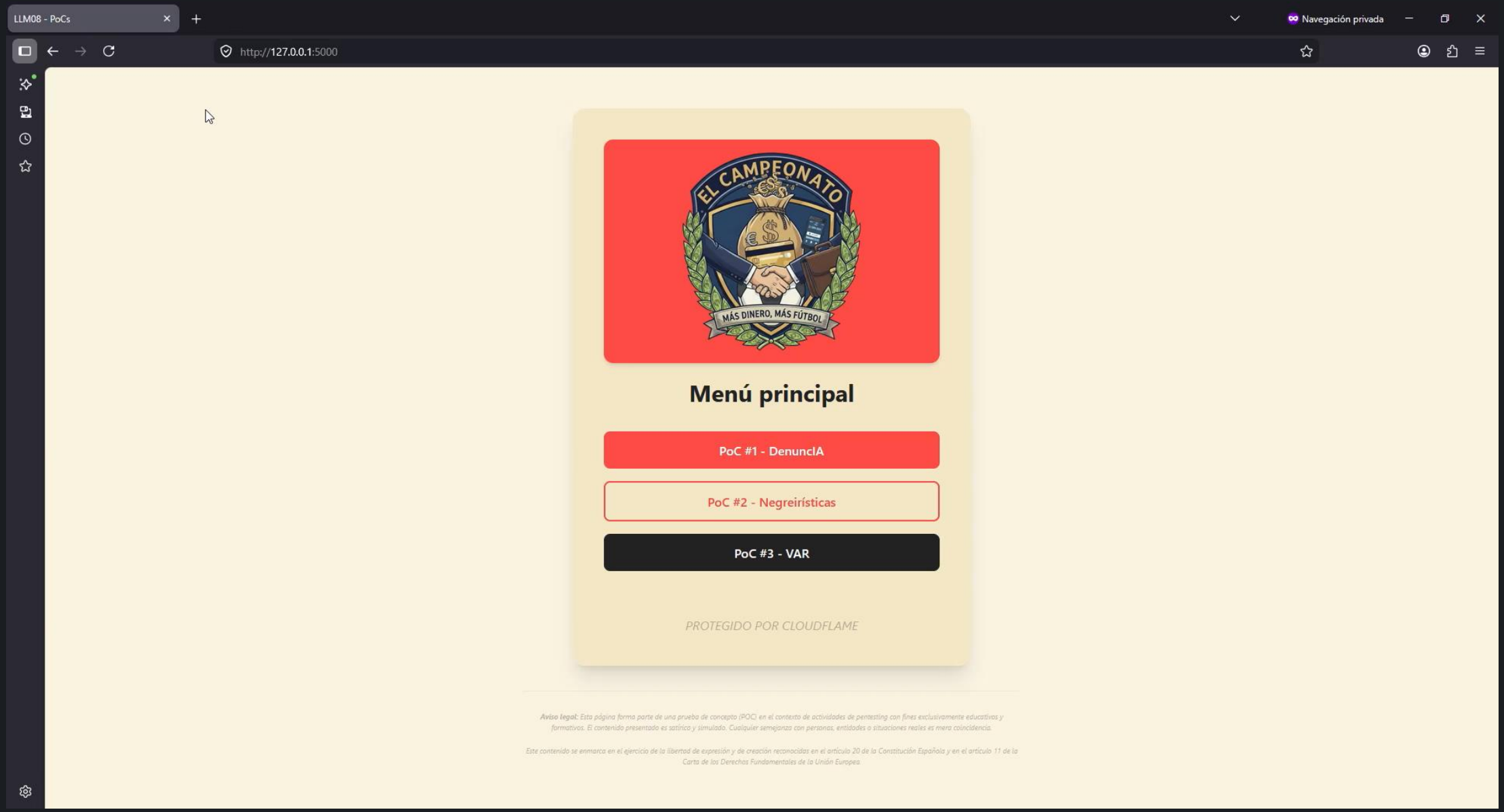
PoC #2: Evasión de Filtros Semánticos



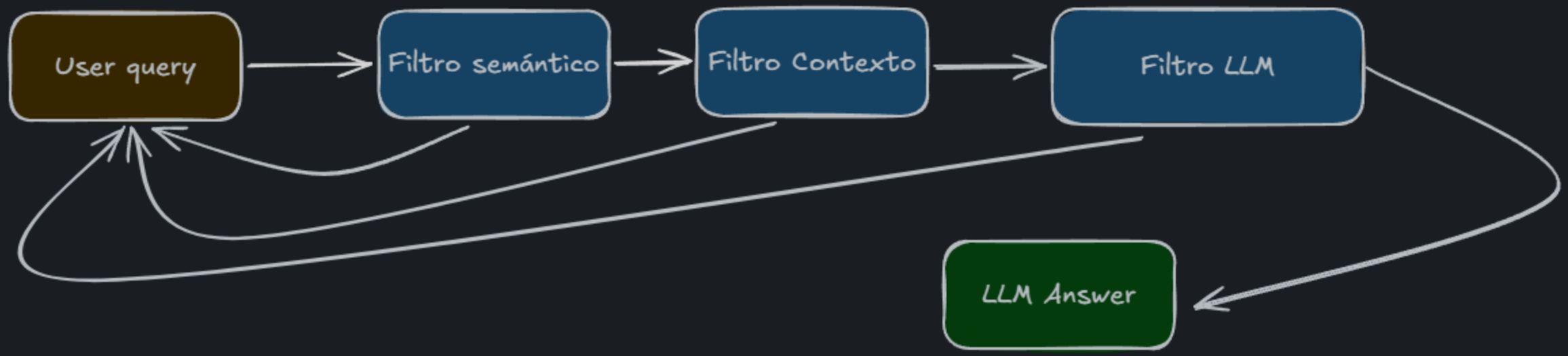
PoC #2: Evasión de Controles Semánticos



PoC #2: Ataque


LLM08 - PoCs
Navegación privada
http://127.0.0.1:5000
EL CAMPEONATO
MÁS DINERO, MÁS FÚTBOL
Menú principal
PoC #1 - DenuncIA
PoC #2 - Negreirísticas
PoC #3 - VAR
PROTEGIDO POR CLOUDFLAME
Aviso legal: Esta página forma parte de una prueba de concepto (POC) en el contexto de actividades de pentesting con fines exclusivamente educativos y formativos. El contenido presentado es satírico y simulado. Cualquier semejanza con personas, entidades o situaciones reales es mera coincidencia.
Este contenido se enmarca en el ejercicio de la libertad de expresión y de creación reconocidas en el artículo 20 de la Constitución Española y en el artículo 11 de la Carta de los Derechos Fundamentales de la Unión Europea.

PoC #2: Evasión de Controles Semánticos



PoC #2: ¿Dónde está la vulnerabilidad?

Los filtros actuan como defensa parcial pero no controlan ni impiden otros problemas, como alucinaciones o accesos a información no filtrada.

1. Filtros mal configurados

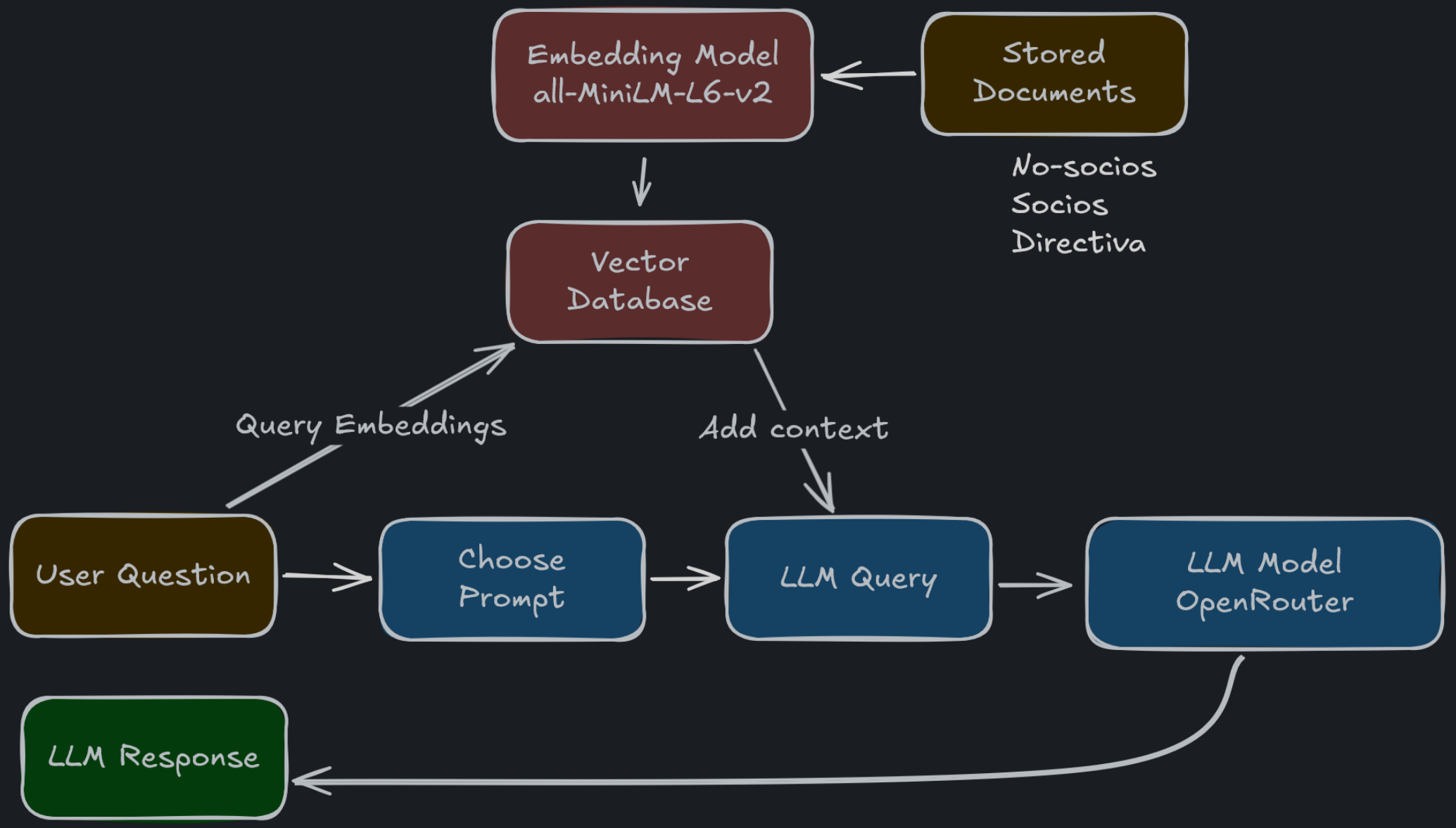
2. Falta de otras defensas

PoC #2: Mitigaciones

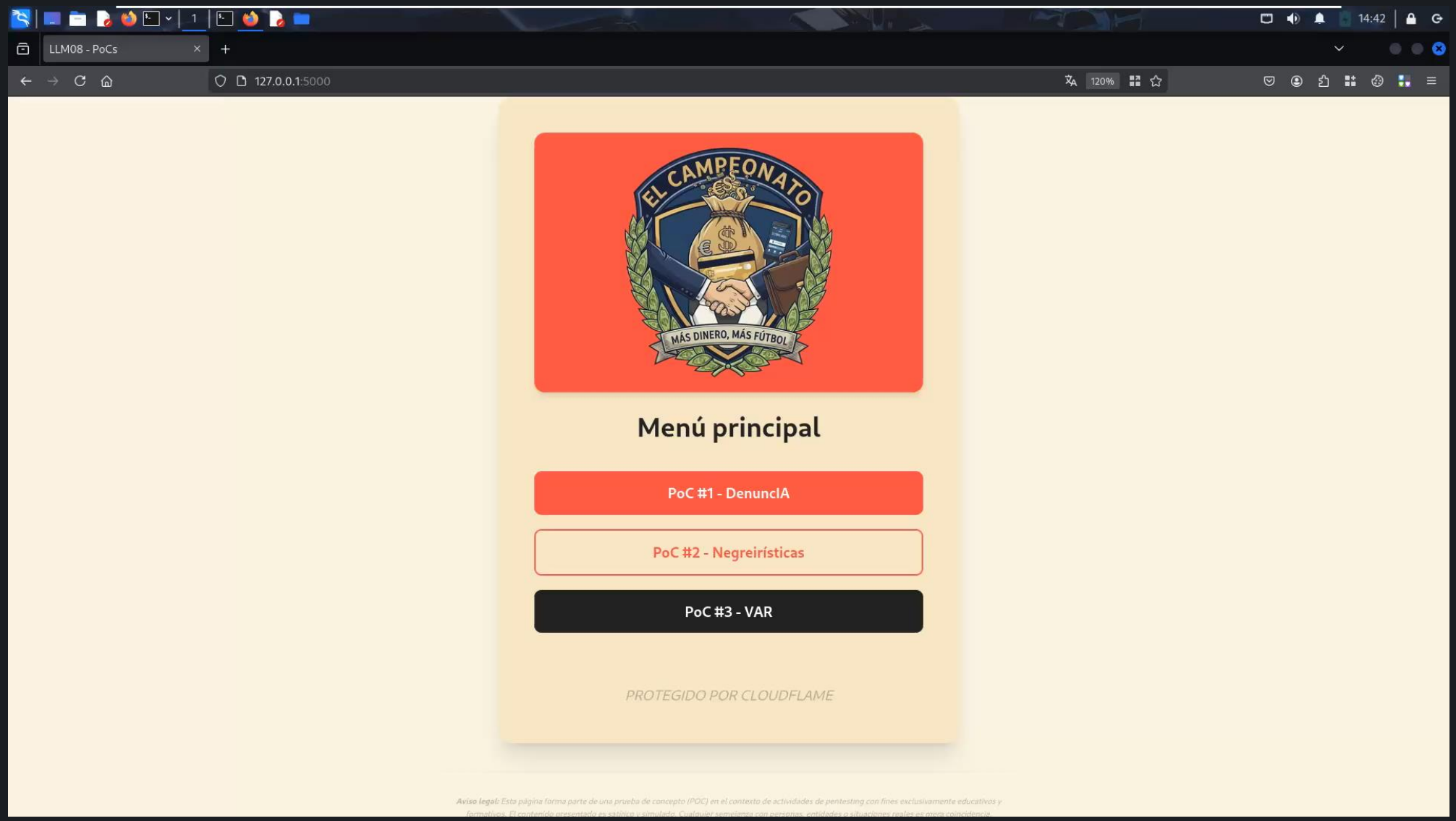
- Filtros Multi-Capa
- Análisis de intención
- Validación de Output
- Human-In-The-Loop

PoC #3: Fallos de Autorización en RAGs

PoC #3: Fallos de Autorización en RAGs - Esquema



PoC #3: Ataque


LLM08 - PoCs
127.0.0.1:5000
120%
14:42
EL CAMPEONATO
MÁS DINERO, MÁS FÚTBOL
Menú principal
PoC #1 - DenuncIA
PoC #2 - Negreirísticas
PoC #3 - VAR
PROTEGIDO POR CLOUDFLAME

PoC #3: ¿Dónde está la vulnerabilidad?

El RAG no distingue entre contenido autorizado o no.
Almacena y recupera la información solicitada.

1. Ausencia de metadatos
2. Confianza en LLM
3. Sin separación lógica

PoC #3: Mitigaciones

- Autorización Pre-Retrieval
- Uso de metadatos
- Separación lógica
- Zero-Trust en el LLM

05

Conclusiones

Patrones inseguros y perspectivas futuras

Recomendaciones de que NO hacer

- IA para pentest de LLM
- Utilizar casos de uso genéricos siempre
- Testear sin contexto del sistema
- Centrarse solo en el Jailbreak

¿Por qué LLM08 va a crecer en relevancia?

- Proliferación de RAG
- Más datos sensibles, mayor impacto
- Maduración de Ataques
- Regulaciones Emergentes

Gracias

¿Preguntas?